

Normative Support Layer

Initiation Gates and Preconditions v0.1

AGL - Runtime Entry Conditions for Grounded Action

Status	Draft v0.1
Layer	Normative support layer / initiation-control discipline
Canonical home	ester-reality-bound
Parent layer	Actor Grounding Layer v0.1
Related layer	Source State Qualification and Runtime Reliance v0.1
Author	Kotov Ivan
Location	Brussels
Year	2026

Kotov Ivan
Bruxelles, 2026

Contents

Abstract	2
1. Purpose	2
2. Scope	2
3. Non-goals	3
4. Core principle	3
5. Why initiation needs its own layer	4
6. Definitions	4
7. Precondition classes	5
8. Gate outcomes	6
9. Gate families	7
10. Human-state preconditions are real, not paternalistic	9
11. Sensor-state preconditions are not optional decoration	10
12. Local entity-state preconditions	10
13. Commit-time revalidation	10
14. Relationship to Source-State Qualification and Runtime Reliance	11
15. Relationship to ARL	11
16. Relationship to L4 Witness and Hardware Perimeter	11
17. Fail-closed rules	12
18. Implementation-facing note	12
19. Explicit bridge	13
20. Hidden bridges	13
21. Earth paragraph	13

Abstract

A system may correctly observe a source, classify its state, and still fail at the most practical point:

it may allow that source to **start something** it should never have been allowed to start.

That is the problem this document addresses.

The Actor Grounding Layer (AGL) already separates:

- source-state qualification,
- and runtime reliance.

This document defines the next step:

how grounded sources are converted into actual runtime entry conditions for action, escalation, review admission, and lawful re-entry.

The key claim is simple:

A serious system should not wait until after movement begins to decide whether movement was lawful. It should gate initiation itself.

That means:

- not every visible source may initiate,
- not every qualified source may initiate the same class of runtime change,
- and not every action surface should open merely because a signal exists.

Initiation is therefore treated as a control boundary, not as a user-experience detail.

1. Purpose

To define:

- what an initiation gate is,
- which kinds of runtime progression require such gates,
- which preconditions must hold before initiation is lawful,
- how different gate classes differ in severity,
- what fail-closed outcomes follow when preconditions are weak or missing,
- how gate discipline interacts with ARL, L4 Witness, and hardware perimeter,
- and how a system prevents abstract, stale, or degraded sources from acquiring action force at the point of entry.

2. Scope

This document applies to initiation of:

- bounded ordinary action,
- privileged or high-impact action,
- escalation and oracle / remote-witness requests,
- admission into review or contradiction handling,
- re-entry and release from restricted states,
- delegated or proxy-origin execution,

- and any runtime path where consequence may begin binding before later review.

It applies across the four source classes already fixed in AGL:

- human / operator state,
- local entity initiation state,
- sensor / perception state,
- delegated / proxy / external origin state.

It applies before and during:

- action attempt,
 - pre-admissibility hold,
 - freeze entry,
 - escalation routing,
 - release checks,
 - and commit-time revalidation.
-

3. Non-goals

This document does **not**:

- replace the full AGL core,
- replace Source-State Qualification and Runtime Reliance,
- replace ARL standing or evidence doctrine,
- replace L4 Witness,
- replace hardware perimeter controls,
- define UI behavior,
- or claim that all safe systems share one universal gate design.

It also does not attempt to settle metaphysical questions about authenticity. Its concern is narrower:

what must be true before a source is allowed to move runtime from possibility into initiated consequence.

4. Core principle

4.1 Initiation is a control boundary

Initiation is not a harmless prelude. It is the moment where a source begins to acquire operational force.

4.2 Preconditions belong before movement, not after embarrassment

A system that only evaluates grounding after initiation has already accepted avoidable risk.

4.3 Different gates require different preconditions

A source may be grounded enough to support review, while being too weak to support privileged action. A source may be grounded enough to request escalation, while being too weak to authorize re-entry.

4.4 Missing preconditions must narrow or stop the path

Where required preconditions are absent, the system must:

- hold,
- narrow,
- reroute,
- deny,
- or quarantine.

It must not compensate with style.

4.5 Some gates must re-check at commit

A path may satisfy preconditions at initiation and fail them later. High-impact progression must therefore support revalidation before consequence binds.

5. Why initiation needs its own layer

A recurring failure pattern in modern systems is this:

1. a source appears coherent,
2. the system lets it open a runtime path,
3. later layers try to repair legality,
4. and by then movement has already produced contamination.

This is common in:

- privilege escalation,
- proxy-origin action,
- stale perception,
- overloaded human initiation,
- drifted local continuity,
- and release from quarantine under optimism.

The problem is not that later review is useless. The problem is that later review often arrives **too late** to prevent the initial widening of force.

That is why initiation gates exist. They are the architecture's answer to a simple question:

should this source be allowed to begin this path at all?

6. Definitions

6.1 Initiation

The first runtime step that gives a source operational influence over:

- action,
- escalation,
- review admission,
- release,
- or another path whose consequence may later bind.

6.2 Gate

A bounded runtime checkpoint that decides whether initiation may proceed, proceed only under narrowed conditions, be rerouted, or be denied.

6.3 Precondition

A required operational fact that must hold before the gate may open for a given initiation class.

6.4 Gate-open

The path may proceed within the defined scope.

6.5 Gate-open-with-limits

The path may proceed only with narrowed authority, shorter windows, stronger logging, stricter witness binding, or other bounded reductions.

6.6 Gate-reroute

The path may not proceed as ordinary execution and must instead be redirected into hold, review, oracle request, contradiction handling, or another bounded alternate flow.

6.7 Gate-deny

The path may not proceed.

6.8 Gate-quarantine

The attempted initiation indicates contamination or illegitimate re-entry strong enough to isolate the relevant branch, artifact, or authority surface.

7. Precondition classes

The following classes of preconditions may be required by one or more initiation gates.

7.1 Source grounding precondition

The source must satisfy the grounding class required for the gate.

7.2 Identity / continuity precondition

The initiating actor or source lineage must still be the correct one for the current operational path.

7.3 Privilege precondition

The source must still hold lawful privilege for the exact action class being initiated. Not earlier. Not nearby. Now.

7.4 Perimeter precondition

The surrounding operational posture must be valid. Examples:

- correct hardware mode,
- no active anomaly breaker,
- required network state,
- no invalid maintenance posture,
- no active perimeter contradiction.

7.5 Time / window precondition

The required time window must be open and not expired.

7.6 Budget precondition

The path must remain within:

- action budget,
- energy budget,
- spend budget,
- escalation budget,
- and any bounded retry budget.

7.7 Evidence / witness precondition

Where initiation depends on proof-bearing surfaces, minimum evidence or witness commitments must exist before movement begins.

7.8 Quarantine / blocked-state precondition

The target path must not already be under a blocking state that prohibits this initiation class.

7.9 Human-state precondition

Where human initiation matters, the human state must not be too degraded for the gate class.

7.10 Environmental precondition

Current L4 conditions must still support the gate, including current load, thermal posture, maintenance posture, and time-sensitivity of the environment.

8. Gate outcomes

Gate evaluation should resolve into a typed bounded result, not into poetic confidence.

8.1 OPEN

All required preconditions are satisfied strongly enough for this gate class.

8.2 OPEN_WITH_LIMITS

The path may proceed only under narrowed authority, shorter window, stronger logging, reduced action scope, or stronger revalidation.

8.3 HOLD

The path must pause pending stronger grounding, additional evidence, or clarification.

8.4 REROUTE_TO_REVIEW

The path may not proceed as ordinary execution and must enter a bounded review path.

8.5 DENY

The path is not allowed to begin.

8.6 QUARANTINE

The attempted initiation indicates a serious enough legitimacy or contamination problem that the relevant branch or surface must be isolated.

8.7 REVALIDATE_AT_COMMIT

The path may provisionally proceed in narrow form, but may not bind consequence until preconditions are checked again at commit.

9. Gate families

Different initiation classes require different gates. They should not be flattened into one generic “allowed / not allowed” switch.

9.1 Ordinary bounded action gate

Purpose: allow low-to-medium consequence action only when the source is grounded enough for ordinary progression.

Typical minimum preconditions:

- source grounding strong enough for ordinary action,
- valid continuity / identity,
- valid privilege,
- no active blocking state,
- valid local perimeter posture,
- no obvious stale-context failure.

Typical outcomes:

- OPEN
- OPEN_WITH_LIMITS
- HOLD
- DENY

9.2 Privileged action gate

Purpose: control initiation of actions that widen consequence significantly.

Typical minimum preconditions:

- stronger grounding than ordinary action,
- stronger privilege validity,
- valid time window,
- valid budget,
- no active contradiction on lineage or authority,
- witness-ready initiation record where required.

Typical outcomes:

- OPEN_WITH_LIMITS
- HOLD
- REROUTE_TO_REVIEW
- DENY
- sometimes REVALIDATE_AT_COMMIT

9.3 Escalation / oracle gate

Purpose: control whether a source may initiate remote witness use, oracle assistance, or escalation beyond local bounded handling.

Typical minimum preconditions:

- local basis is insufficient but not void,
- explicit escalation reason exists,
- source remains grounded enough to request escalation,
- oracle or remote window is lawful and open,
- spend / call budget permits it,
- no hidden fallback behavior is being attempted.

Typical outcomes:

- OPEN_WITH_LIMITS
- HOLD
- REROUTE_TO_REVIEW
- DENY

9.4 Review-admission gate

Purpose: control whether a source may initiate procedural review rather than ordinary action.

This gate is important because not every weak source should act, but some weak sources may still be strong enough to justify hold or review.

Typical minimum preconditions:

- valid or at least review-eligible source grounding,
- minimum standing basis where applicable,
- non-empty conflict basis,
- enough continuity to identify scope,
- enough witnessability or evidence commitment to avoid pure rhetorical drift.

Typical outcomes:

- REROUTE_TO_REVIEW
- HOLD
- DENY

9.5 Re-entry / release gate

Purpose: control whether a previously held, frozen, or quarantined path may begin movement back toward ordinary flow.

This is usually the strictest gate because re-entry is where illegitimate continuity often launders itself into normal state.

Typical minimum preconditions:

- no unresolved blocking contradiction,
- release conditions satisfied,
- valid current perimeter posture,
- no fresh anomaly or degradation,
- no expired authority basis,
- valid revalidation at commit,

- witness-ready release trace.

Typical outcomes:

- OPEN_WITH_LIMITS
- REVALIDATE_AT_COMMIT
- HOLD
- DENY
- QUARANTINE

9.6 Delegated / proxy-origin gate

Purpose: control initiation from delegated, proxied, provider-mediated, or otherwise abstracted source surfaces.

Typical minimum preconditions:

- delegation remains valid now,
- proxy is sufficiently grounded to live execution,
- no excessive abstraction from the commit boundary,
- no identity ambiguity,
- no policy drift between delegator and active proxy state.

Typical outcomes:

- OPEN_WITH_LIMITS
- HOLD
- REROUTE_TO_REVIEW
- DENY

10. Human-state preconditions are real, not paternalistic

Human sources are not disqualified from grounding logic. They are one of its most important reasons for existence.

A human may remain:

- the owner,
- the anchor,
- the responsible actor,

and still be too degraded **in this moment** for full initiation force.

This includes:

- fatigue,
- overload,
- panic,
- sleep deprivation,
- unstable attention,
- and physiological or cognitive conditions that make the current initiation boundary unreliable.

This is not disrespect. This is anti-catastrophic hygiene.

A mature system should preserve human authority by refusing to translate every immediate human state into full execution power.

11. Sensor-state preconditions are not optional decoration

A perceptual path is not action-worthy merely because input exists.

A sensor-origin initiation may fail gate preconditions because of:

- staleness,
- degraded calibration,
- acquisition gaps,
- broken chain integrity,
- environmental corruption,
- or mismatch between sensor timing and execution timing.

This matters because many systems still treat:

visible input = usable input = actionable input.

AGL gate discipline rejects that collapse.

12. Local entity-state preconditions

A persistent local entity may satisfy identity continuity and still fail initiation preconditions.

Examples:

- unresolved drift,
- overload under review pressure,
- degraded internal topology,
- active deadlock,
- unresolved contradiction,
- narrowed confidence under current L4 pressure.

A system must be able to say:

the continuity remains the same, but the present state is not grounded enough to open this gate.

That is painful. It is also one of the marks of a serious system.

13. Commit-time revalidation

Some initiation gates are too weak if checked only once.

A source may be valid at intake and invalid at commit because:

- human state degrades,
- privilege expires,
- sensor freshness decays,
- perimeter changes,
- anomaly appears,
- time window closes,
- or environmental conditions shift.

Therefore, where consequence is high enough, initiation should not be treated as sufficient by itself.

The path may require:

- OPEN_WITH_LIMITS,
- followed by REVALIDATE_AT_COMMIT.

This is not hesitation. It is preventing stale legitimacy from becoming live consequence.

14. Relationship to Source-State Qualification and Runtime Reliance

This document does not decide grounding dimensions by itself. That remains upstream.

It consumes the result of source-state qualification and converts it into one practical question:

which gate, if any, may open now?

That is why the layers remain distinct:

- **Source-State Qualification** asks what condition the source is in.
- **Runtime Reliance** asks what force the source may carry.
- **Initiation Gates and Preconditions** ask whether that force is strong enough to begin this class of runtime movement now.

Separation matters. Otherwise a system drifts into the lazy shortcut:

qualification -> confidence -> movement

with no explicit control boundary in between.

15. Relationship to ARL

ARL begins when conflict enters procedural form. This document begins earlier.

ARL asks:

- who has standing,
- what evidence is admissible,
- what outcome is lawful,
- what remains frozen,
- what may re-enter.

This document asks:

- can this source open an action path,
- can it open a review path,
- can it request escalation,
- can it support release,
- or must it be held, denied, rerouted, or quarantined before ARL becomes the active layer.

AGL therefore reduces the burden on ARL by stopping weak or degraded initiation earlier.

16. Relationship to L4 Witness and Hardware Perimeter

16.1 Witness

L4 Witness records what occurred. It may also record gate outcomes. But witness alone does not decide whether a gate should open.

16.2 Hardware perimeter

Hardware perimeter supplies concrete grounding inputs such as:

- mode,
- anomaly flags,
- power posture,
- interface posture,
- maintenance state,
- operator attestation,
- and circuit-breaker state.

This document consumes those as gate preconditions.

In other words:

- Witness records.
- Perimeter conditions.
- Initiation gates decide whether runtime may begin moving.

17. Fail-closed rules

17.1 No precondition, no initiation

A missing required precondition must not be papered over by coherence.

17.2 Review is not a loophole for ordinary action

A weak source may be strong enough to justify review, but that does not make it strong enough to justify ordinary execution.

17.3 Re-entry is stricter than entry

Release and re-entry should generally require stronger preconditions than ordinary initiation.

17.4 Delegation does not preserve grounding automatically

A proxied or delegated path may remain structurally valid while becoming operationally too abstract to trust.

17.5 Commit may still refuse

Even after a gate opens provisionally, commit-time revalidation may still close the path.

18. Implementation-facing note

This document naturally maps to implementation surfaces already visible in the ECC bridge package:

- action-attempt intake hooks,
- hold-entry hooks,
- privilege narrowing hooks,

- oracle window gates,
- review-routing hooks,
- re-entry legality hooks,
- witness emission,
- and dispute-state persistence.

Its job is not to define all code paths. Its job is to ensure that those runtime hook points consume a serious precondition model rather than generic “allowed / denied” folklore.

19. Explicit bridge

source-state qualification -> initiation gates -> runtime reliance -> ARL / release / denial discipline

This document is the control boundary that prevents grounded observation from being mistaken for lawful initiation.

20. Hidden bridges

20.1 Cybernetics / Ashby

A regulator that only reacts after motion has already widened force is often too late. Initiation gates add the requisite variety needed to damp action before escalation turns into contamination.

20.2 Information theory / bounded channels

Not every signal deserves equal action bandwidth. Initiation gates compress operational force by admitting only grounded sources into stronger channels of runtime consequence.

21. Earth paragraph

In a real operating room, the cut does not begin because someone in the room sounds confident. Identity is checked, consent is checked, instruments are checked, the patient state is checked, the team state is checked, and if something is wrong the incision does not start. That is not bureaucracy. It is the difference between preparation and damage. Initiation gates play the same role in long-lived digital systems: before consequence begins, the system checks whether this path is actually grounded enough to be allowed to move.